
title: "Práctica: construye tu mini-motor DLP" subtitle: "Detecta y protege datos sensibles escribiendo, paso a paso, el mismo tipo de motor que usan las herramientas DLP profesionales" tags: ["DLP", "ciberseguridad", "python", "datos personales"] authors: ["jalvarez13"]

Práctica: construye tu mini-motor DLP

En el módulo has visto **qué** hace una solución DLP: encontrar datos sensibles y evitar que se escapen. En esta práctica vas a ver **cómo** lo hace por dentro. Vas a escribir, desde cero y en Python, un programa que recorre los ficheros de una empresa ficticia, encuentra tarjetas de crédito, cuentas bancarias, DNI, correos y teléfonos, los clasifica por sensibilidad y genera un informe.

No necesitas saber programar. Cada paso explica qué se escribe y por qué. Si en algún momento te atascas, tienes el programa terminado en la carpeta [solution/](#) para compararlo con el tuyo.

El código va en inglés (nombres de variables, funciones y campos), que es la convención habitual en la industria. Las explicaciones están en español.

⚠ Todos los datos del ejercicio son **inventados**. Las tarjetas y los DNI que usamos son números de prueba que cumplen el formato pero no pertenecen a nadie. Nunca trabajes con datos reales de personas para practicar.

Qué vas a construir

Al terminar, tu programa hará esto:

```
$ python mini_dlp.py --path ../sample-data

JSON report written to report.json

Total findings: 30
By sensitivity level:
  - HIGH: 14
  - MEDIUM: 16
By data type:
  - API_TOKEN: 1
  - CREDIT_CARD: 3
  - DNI_NIE: 5
  - EMAIL: 10
  - IBAN: 5
  - PHONE_ES: 6
```

Y generará un fichero `report.json` con cada hallazgo, su ubicación y el valor **enmascarado** (`45** *67`), nunca el dato completo.

Lo que vas a aprender

- Cómo un motor DLP localiza datos sensibles con **expresiones regulares**.
- Por qué la forma de un dato no basta, y cómo se **valida** una tarjeta con el algoritmo de Luhn, un IBAN con mod-97 y un DNI con su letra de control.

- Cómo se **clasifica** la información por sensibilidad (lo que viste en teoría).
- Cómo se **enmascara** un dato para registrarlo sin exponerlo.
- En qué se diferencia tu motor de una herramienta profesional (Microsoft Presidio).

Antes de empezar

Necesitas dos cosas:

1. **Python 3.10 o superior**. Compruébalo abriendo una terminal y escribiendo:

```
bash python3 --version
```

Si ves algo como **Python 3.12.1**, ya lo tienes. Si da error, instálalo desde python.org/downloads (marca la casilla "Add Python to PATH" durante la instalación en Windows).

2. **Un editor de código**. Vale [Visual Studio Code](https://code.visualstudio.com/), que es gratuito.

Descarga también esta carpeta del proyecto. Dentro tienes **sample-data/**, que simula los ficheros de una empresa, y la carpeta **solution/** con el resultado final por si quieres consultarlo.

Parte 0 — Prepara tu carpeta de trabajo

Crea un fichero vacío llamado **mini_dlp.py** dentro de la carpeta del proyecto, al lado de **sample-data/**. Ahí irás escribiendo el código de cada parte.

Para ejecutarlo, abre la terminal **en esa carpeta** y usa:

```
python3 mini_dlp.py
```

Cada vez que termines una parte, ejecuta el programa para ver cómo avanza.

Parte 1 — Cómo razona un motor DLP

Un motor DLP de contenido repite cuatro pasos con cada texto que revisa:

1. **Buscar** trozos que *parezcan* un dato sensible (un patrón).
2. **Validar** que ese trozo lo es de verdad (descartar falsas alarmas).
3. **Clasificar** el dato según lo sensible que sea.
4. **Actuar**: registrarlo, enmascararlo, avisar o bloquear.

Tu programa va a hacer exactamente eso. Empezamos por el paso 1.

Parte 2 — Tu primer detector: el correo electrónico

Antes de tocar el programa grande, prueba algo pequeño para ver Python en marcha. Una **expresión regular** (o *regex*) es una fórmula para describir un patrón de texto. Esta describe un correo: "letras o números, una arroba, un dominio y una extensión".

Escribe esto en **mini_dlp.py** y ejecútalo:

```
import re

text = "Escríbeme a ana.garcia@innova.es cuando puedas."

pattern = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
print(pattern.findall(text))
```

Al ejecutar `python3 mini_dlp.py` verás:

```
['ana.garcia@innova.es']
```

Acabas de escribir tu primer detector. `re.compile` prepara el patrón y `findall` devuelve todo lo que encaja. Esa `r` antes de las comillas le dice a Python que no interprete las barras invertidas: son parte de la regex.

💡 No memorices la regex del correo. Lo importante es entender que un patrón describe la *forma* de un dato. La forma exacta la copias de referencias fiables y la ajustas.

Parte 3 — Un dato con trampa: la tarjeta de crédito

Abre el fichero `sample-data/support/ticket-4521.txt`. Verás dos tarjetas y, más abajo, esta línea:

```
Verificado el cargo con el número de pedido 1234 5678 9012 3456 (esto NO es una tarjeta, es el identificador interno del pedido).
```

Ese número de pedido tiene 16 cifras, igual que una tarjeta. Si solo miras la forma, lo marcarías como tarjeta y te equivocarías. Eso es un **falso positivo**, y en DLP son un problema serio: si tu sistema da demasiadas falsas alarmas, la gente deja de hacerle caso.

Las tarjetas reales cumplen el **algoritmo de Luhn**: una fórmula que usa la última cifra como dígito de control. El número de pedido no lo cumple. Esta función comprueba Luhn:

```
def is_valid_luhn(number: str) -> bool:
    digits = [int(c) for c in number if c.isdigit()]
    if len(digits) < 13:
        return False
    checksum = 0
    parity = len(digits) % 2
    for i, digit in enumerate(digits):
        if i % 2 == parity:
            digit *= 2
            if digit > 9:
                digit -= 9
        checksum += digit
    return checksum % 10 == 0
```

Recorre las cifras, duplica una de cada dos (restando 9 si pasa de 9), las suma y comprueba que el total es múltiplo de 10. Pruébala añadiendo al final del fichero:

```
print(is_valid_luhn("4539 1488 0343 6467")) # real test card -> True
print(is_valid_luhn("1234 5678 9012 3456")) # order number -> False
```

Esta es la idea clave de la práctica: **patrón para encontrar, validación para confirmar**.

Parte 4 — Datos españoles: DNI/NIE e IBAN

Los datos de tus ficheros son españoles, así que necesitas dos validadores más.

El **DNI** termina en una letra que se calcula con el resto de dividir el número entre 23. El **NIE** es igual, pero empieza por X, Y o Z (que valen 0, 1 y 2):

```
DNI_LETTERS = "TRWAGMYFPDXBNJZSQVHLCKE"
NIE_PREFIX = {"X": "0", "Y": "1", "Z": "2"}

def is_valid_dni(value: str) -> bool:
    text = value.upper().replace("-", "").replace(" ", "")
    if len(text) < 9:
        return False
    if text[0] in NIE_PREFIX:
        number = NIE_PREFIX[text[0]] + text[1:8]
        letter = text[8]
    else:
        number = text[:8]
        letter = text[8]
    if not number.isdigit() or not letter.isalpha():
        return False
    return DNI_LETTERS[int(number) % 23] == letter
```

El **IBAN** (la cuenta bancaria) se valida con mod-97: mueves el código de país y los dos dígitos de control al final, cambias las letras por números (A=10, B=11... Z=35) y compruebas que el número gigante resultante da resto 1 al dividir entre 97:

```
def is_valid_iban(value: str) -> bool:
    iban = value.upper().replace(" ", "")
    if len(iban) < 15 or not iban[:2].isalpha():
        return False
    rearranged = iban[4:] + iban[:4]
    converted = ""
    for char in rearranged:
        if char.isdigit():
            converted += char
        elif char.isalpha():
            converted += str(ord(char) - 55)
        else:
            return False
    return int(converted) % 97 == 1
```

No tienes que memorizar estas fórmulas. Son estándares públicos (Luhn, ISO 13616 para el IBAN) y aquí las usas como lo haría una herramienta real: para no dar falsas alarmas.

Parte 5 — Reúne todos los detectores

Ahora juntas patrón, validador y nivel de sensibilidad en una sola lista. Cada detector es un diccionario. Los que tienen `"validator": None` solo miran la forma (un correo no tiene dígito de control); los demás validan.

```

DETECTORS = [
    {
        "type": "CREDIT_CARD",
        "label": "Credit card",
        "regex": re.compile(r"\b(?:\d[ -]){13,19}\b"),
        "validator": is_valid_luhn,
        "sensitivity": "HIGH",
    },
    {
        "type": "IBAN",
        "label": "Bank account (IBAN)",
        "regex": re.compile(r"\b[A-Z]{2}\d{2}(?:[ ]?\d){10,30}\b"),
        "validator": is_valid_iban,
        "sensitivity": "HIGH",
    },
    {
        "type": "DNI_NIE",
        "label": "Spanish national ID (DNI/NIE)",
        "regex": re.compile(r"\b[XYZ]?[0-9]{7,8}[- ]?[A-Za-z]\b"),
        "validator": is_valid_dni,
        "sensitivity": "HIGH",
    },
    {
        "type": "API_TOKEN",
        "label": "API token / key",
        "regex": re.compile(r"\b(?:sk|pk|ghp|xoxb)[-_] [A-Za-z0-9_-]{8,}\b"),
        "validator": None,
        "sensitivity": "HIGH",
    },
    {
        "type": "EMAIL",
        "label": "Email address",
        "regex": re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"),
        "validator": None,
        "sensitivity": "MEDIUM",
    },
    {
        "type": "PHONE_ES",
        "label": "Spanish phone number",
        "regex": re.compile(r"(?!\\d)(?:\\+34[ ])?[6789]\\d{2}[ ]?\\d{3}[ ]?\\d{3}(?!\\d)"),
        "validator": None,
        "sensitivity": "MEDIUM",
    },
]

```

La función que recorre un texto aplica cada detector, descarta lo que no valida y resuelve los solapamientos (cuando dos patrones pillan el mismo trozo, se queda con el más largo):

```
def scan_text(text: str) -> list[dict]:
    candidates = []
    for detector in DETECTORS:
        for match in detector["regex"].finditer(text):
            value = match.group()
            validator = detector["validator"]
            if validator is not None and not validator(value):
                continue
            candidates.append({
                "type": detector["type"],
                "label": detector["label"],
                "value": value,
                "sensitivity": detector["sensitivity"],
                "start": match.start(),
                "end": match.end(),
            })

    candidates.sort(key=lambda c: (c["start"], -(c["end"] - c["start"])))
    findings = []
    last_end = -1
    for candidate in candidates:
        if candidate["start"] >= last_end:
            findings.append(candidate)
            last_end = candidate["end"]
    return findings
```

Parte 6 — Enmascara antes de registrar

Un motor DLP nunca guarda el dato completo en sus informes: lo enmascara. Esta función deja a la vista los dos primeros y dos últimos caracteres y tapa el resto con asteriscos:

```
def redact_value(value: str) -> str:
    chars = list(value)
    alnum = [i for i, c in enumerate(chars) if c.isalnum()]
    if len(alnum) > 4:
        for i in alnum[2:-2]:
            chars[i] = "*"
    else:
        for i in alnum:
            chars[i] = "*"
    return "".join(chars)
```

Así, **4539 1488 0343 6467** se registra como **45** **** **67** : suficiente para identificar el hallazgo, imposible de reutilizar.

Parte 7 — Recorre los ficheros de la empresa

Hasta ahora trabajabas con un texto suelto. Ahora lees ficheros de verdad. Estas funciones abren cada fichero de una carpeta (y sus subcarpetas), lo analizan y anotan en qué línea está cada hallazgo:

```

from pathlib import Path

def line_of(text: str, position: int) -> int:
    return text.count("\n", 0, position) + 1

def scan_path(root: Path) -> list[dict]:
    files = [root] if root.is_file() else sorted(p for p in root.rglob("*") if p.is_file())
    findings = []
    for file in files:
        try:
            text = file.read_text(encoding="utf-8", errors="replace")
        except (OSError, UnicodeError):
            continue
        for hit in scan_text(text):
            findings.append({
                "file": str(file),
                "line": line_of(text, hit["start"]),
                "type": hit["type"],
                "label": hit["label"],
                "sensitivity": hit["sensitivity"],
                "masked_value": redact_value(hit["value"]),
            })
    return findings

```

Añade `from pathlib import Path` arriba del todo, junto a `import re`.

Parte 8 — Clasifica y genera el informe

Por último, cuentas los hallazgos por tipo y por nivel, y los guardas en JSON y CSV. La clasificación por sensibilidad (HIGH/MEDIUM) es la misma idea que viste en la teoría de clasificación de datos.

```

import csv
import json

def summarize(findings: list[dict]) -> dict:
    by_type: dict[str, int] = {}
    by_level: dict[str, int] = {}
    for f in findings:
        by_type[f["type"]] = by_type.get(f["type"], 0) + 1
        by_level[f["sensitivity"]] = by_level.get(f["sensitivity"], 0) + 1
    return {"total": len(findings), "by_type": by_type, "by_level": by_level}

def write_json(findings: list[dict], summary: dict, path: Path) -> None:
    data = {"summary": summary, "findings": findings}
    path.write_text(json.dumps(data, indent=2, ensure_ascii=False), encoding="utf-8")

def write_csv(findings: list[dict], path: Path) -> None:
    columns = ["file", "line", "type", "label", "sensitivity", "masked_value"]
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=columns)
        writer.writeheader()
        writer.writerows(findings)

```

Y el bloque final que une todo y permite ejecutarlo desde la terminal con opciones (`--path`, `--format`, `--output`):

```
import argparse

def main() -> None:
    parser = argparse.ArgumentParser(description="Example mini DLP engine.")
    parser.add_argument("--path", default="../sample-data")
    parser.add_argument("--format", choices=["json", "csv", "both"], default="json")
    parser.add_argument("--output", default="report")
    args = parser.parse_args()

    root = Path(args.path)
    if not root.exists():
        parser.error(f"Path '{root}' does not exist.")

    findings = scan_path(root)
    summary = summarize(findings)

    if args.format in ("json", "both"):
        write_json(findings, summary, Path(f"{args.output}.json"))
        print(f" JSON report written to {args.output}.json")
    if args.format in ("csv", "both"):
        write_csv(findings, Path(f"{args.output}.csv"))
        print(f" CSV report written to {args.output}.csv")

    print(f"\n Total findings: {summary['total']}")
    print(f" By level: {summary['by_level']}")
    print(f" By type: {summary['by_type']}")

if __name__ == "__main__":
    main()
```

Ejecuta la versión completa apuntando a la carpeta de datos:

```
python3 mini_dlp.py --path ../sample-data --format both
```

Abre `report.json` y comprueba dos cosas:

- El número de pedido `1234 5678 9012 3456` **no** aparece como tarjeta. Tu validador de Luhn lo descartó.
- Ninguna tarjeta, DNI o IBAN aparece con el valor completo: todos van enmascarados.

Si te sale lo mismo, tu motor DLP funciona. Compara tu fichero con [solution/mini_dlp.py](#) si algo no cuadra.

Parte 9 (opcional) — Compáralo con una herramienta profesional

[Microsoft Presidio](#) es una librería DLP de código abierto (licencia MIT). Además de patrones, usa un modelo de lenguaje para reconocer nombres de personas, lugares u organizaciones, algo que una regex no puede hacer. Vas a analizar el mismo texto con tu motor y con Presidio.

Primero crea un entorno virtual e instala las dependencias:


```
python3 -m venv .venv
source .venv/bin/activate          # en Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m spacy download es_core_news_sm
```

Después ejecuta el script de comparación que tienes en `solution/`:

```
python solution/compare_presidio.py
```

Verás algo así:

```
Homemade mini engine detects:
['CREDIT_CARD', 'DNI_NIE', 'EMAIL', 'PHONE_ES']

Microsoft Presidio detects:
['CREDIT_CARD', 'EMAIL_ADDRESS', 'ES_NIF', 'LOCATION', 'PERSON', 'PHONE_NUMBER', 'URL']
```

Fíjate en la diferencia: los dos aciertan con la tarjeta, el correo, el teléfono y el DNI. Pero Presidio detecta además **PERSON** ("Diego Fernández") y **LOCATION** ("Sevilla"), porque entiende el significado de las palabras, no solo su forma.

La conclusión que debes sacar:

- Para datos con estructura fija (tarjeta, IBAN, DNI), un motor de reglas como el tuyo es rápido, transparente y fácil de auditar.
- Para datos sin forma fija (nombres, direcciones), hacen falta modelos de lenguaje. Las herramientas DLP serias combinan los dos enfoques.

Entregables

Sube a tu repositorio:

1. Tu fichero `mini_dlp.py` funcionando.
2. El `report.json` (o `report.csv`) generado sobre `sample-data/`.
3. Un `ANSWERS.md` corto (media página) que responda: - ¿Cuántos datos de sensibilidad HIGH encontró tu programa? - ¿Por qué el número de pedido `1234 5678 9012 3456` no se marcó como tarjeta? - Si tuvieras que aplicar un control DLP a estos ficheros, ¿cuál pondrías y en qué carpeta? Relaciónalo con la teoría del módulo.

Rúbrica de evaluación

Criterio	Puntos
El programa se ejecuta sin errores sobre <code>sample-data/</code>	25
Detecta correctamente correos, teléfonos, DNI, IBAN y tarjetas	25
Los validadores (Luhn, mod-97, letra del DNI) descartan los falsos positivos	20
Los hallazgos se guardan enmascarados , nunca en claro	15
El <code>ANSWERS.md</code> conecta el ejercicio con la teoría de DLP	15
Total	100

Retos para subir nota

- Añade un detector de **IP** y otro de **fecha de nacimiento**.
 - Adapta `redact_value` para que el correo se enmascare como `****@innova.es`.
 - Añade una opción `--only-high` que filtre y muestre únicamente los hallazgos de sensibilidad HIGH.
 - Genera un pequeño resumen en HTML, además del JSON.
-

Problemas frecuentes

- **python3: command not found**: prueba con `python` en lugar de `python3`. En Windows, reinstala Python marcando "Add Python to PATH".
- **IndentationError**: en Python los espacios al principio de línea importan. Usa siempre 4 espacios, nunca tabuladores mezclados.
- **No detecta nada**: comprueba que ejecutas el programa desde la carpeta correcta y que la ruta de `--path` apunta a `sample-data`.
- **ModuleNotFoundError: presidio_analyzer** (Parte 9): no has activado el entorno virtual o no instalaste las dependencias. Repite los pasos de la Parte 9. Recuerda que la Parte 9 es opcional; las Partes 1 a 8 no necesitan instalar nada.

Por qué esta práctica importa

Acabas de construir, en miniatura, lo que hace el motor de contenido de cualquier DLP: encontrar, validar, clasificar y proteger. Cuando configures una herramienta profesional, ya sabrás qué ocurre por debajo de la interfaz, por qué aparecen falsos positivos y por qué la clasificación de datos es el primer paso de todo programa DLP.